

Smart Array Optimizer (SAO)

במדריך הבא אסביר איך להשתמש ב-GUI המאטלב שעושה אופטימיזציה למשקלים ופאזות בין אלמנטים שונים במערך לא טריוויאלי.

עיקרי הדברים:

האלגוריתם שעומד בלב האופטימיזציה הוא L-BFGS-B עבור גרסת הפייתון ו-fmincon עבור גרסת המאטלב. אלו שיטות אופטימיזציה אחרות, אבל התוצאות המתקבלות זהות (גם אם לעיתים זמני ההתכנסות שונים).

בגרסת המאטלב, האלגוריתם משערך את ההסיאן (Hessian) בנקודה שהוא נמצא בו, פותר את המינימום בהנחה שהפונקציה convex וצועד לשם (די דומה למורד הגרדיאנט מסדר שני). בבעיות פשוטות ההתכנסות שלו מאד מהירה.

עקום הקרינה מחושב באופן הבא:

$$AF(\theta, \phi) = \sum_n w_n E_n(\theta, \phi)$$

כאשר w_n הוא גודל מרוכב המתאר את האמפליטודה והפאזה היחסית עבור האלמנט ה-n, ו- $E_n(\theta, \phi)$ הוא עקום הקרינה של האלמנט ה-n, כפי שהתקבל מ-CST.

האופטימיזציה היא על מינימיזציה של הגודל הבא:

$$J = \sum_k \lambda_k C_k$$

כאשר λ_k הוא המשקל שניתן באופטימיזציה למשימה ה-k, ו- C_k היא פונקציית המחיר ה-k, שיכולה להיות מינוס, מינימום או מקסימום של ה-AF והסימן חיובי/שלילי – נקבע לפי האם נדרש null או peak.

בשביל לעשות את השימוש בכלי נוח ופשוט ככל האפשר, נבנה GUI שברמה הבסיסית ביותר, אין צורך לצאת ממנו לקוד עצמו.

מטרת המסמך היא לתאר בפשטות אילו עושים בו שימוש.

אז איך עושים את זה?

אני אתאר כאן בקצרה את הדרך של ה-GUI ושל התוכנה

שלב 1 – ייצוא הנתונים

נכנסים ל-CST, בונים את המודל, מריצים ומייצאים (בעזרת result template postprocessing) את עקומי הקרינה ברזולוציה הרצויה. בשלב הזה חשוב לוודא שה-CST נמצא בבסיס הקיטוב הנכון, איפה ראשית הצירים ולאן הצירים מצביעים – כל זה בשביל לוודא שהגדרת המטרה מתאימה לצורך שלכם.

הקבצים ייוצרו לכם בתיקיית פרויקט ה-CST, תחת Export.

אני ממליץ להעביר את התיקייה שנוצרה לתיקיית data בתיקיית הפרויקט (הזה).

שלב 2 – הגדרת מטרה

דרך הקוד

ניתן לפתוח את הקובץ `matlab_config.yaml` שנמצא בתיקייה `MATLAB/scripts` ההגדרות שם די מסבירות את עצמן. בכל זאת אעבור על מה שמופיע שם בקצרה:

1. הדרך לתיקייה שמכילה את עקומי הקרינה
2. בחירת קיטוב (אחד משני הקיטובים שיצאו מהCST או 'total')
3. מטרות (הצורה שבה מגדירים אותם מפורטת מאד בגוף הקובץ)
4. הגדרות אופטימיזר
 - a. מספר צעדים מקסימלי
 - b. כמה הרצות בלתי תלויות לעשות (מומלץ לפחות שניים)
 - c. האם להתחיל גם מהתפלגות אחידה
 - d. האם להתחיל מכל אלמנט בנפרד
 - e. האם האופטימיזציה היא על המשקלים או הפאזות בלבד או גם וגם?
5. הגדרות פלט
 - a. לאן לייצא
 - b. איזה חתכים לייצא
 - c. איך להציג עקום ספרי על דו מימד

דרך הGUI

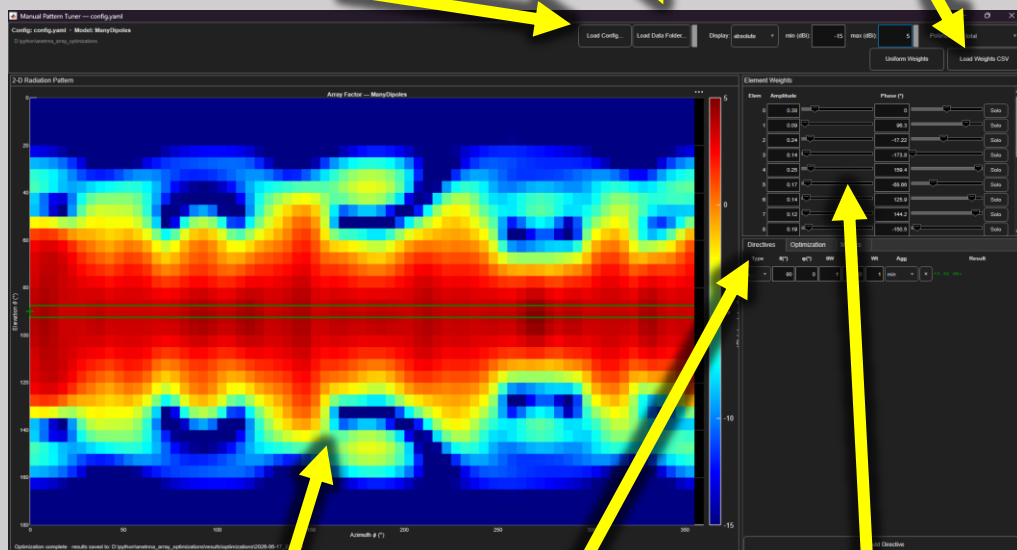
פירוט מלא על כל אחד מהשדות נמצא בתוך הקובץ `matlab_config.yaml` שנמצא בתיקייה `MATLAB/scripts`

פתיחת הGUI נעשית דרך הרצה של הקובץ `\MATLAB\scripts\optimizer_app.m`

טעינת קובץ קונפיגורציה
(משקלים שכבר רצו)

טעינת עקומי קרינה

טעינת קובץ נתונים
(משקלים שכבר רצו)



הצגת עקום הקרינה

חלונות הגדרות של מטרה

שליטה באמפליטודה
ובפאזה של כל אלמנט

שלב 3 – ריצה וצפייה בתוצרים

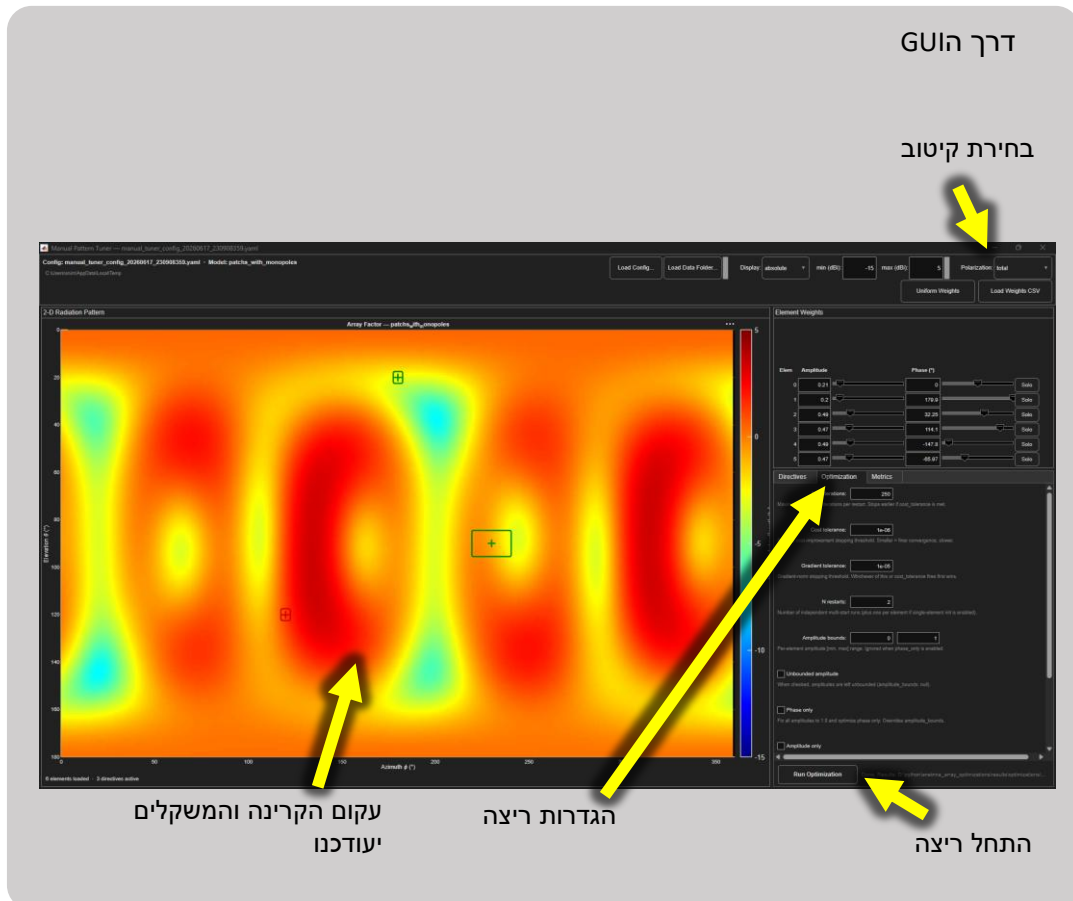
דרך הקוד

יש להריץ את :

```
\MATLAB\scripts\run_optimization_script.m
```

דרך הGUI

בחירת קיטוב



עקום הקרינה והמשקלים
יעודכנו

הגדרות ריצה

התחל ריצה

בשתי השיטות יש פלט command window שמציג את ההתקדמות, שמני ההתכנסות ואיכות התוצר (בשפה של פונקציית המטרה, לא יגיד הרבה כשלעצמו)

תפתח תיקייה עם timestamp של זמן התחלת הריצה בתיקייה:

\results\optimizations\

ושם מרוכזים כלל התוצרים:

- GIF שמתאר את התקדמות האופטימיזציה
- גרף של המשקלים שהתקבלו
- CSV של המשתנים שהתקבלו
- העקום שהתקבל
- קובץ טקסט שמתאר את התוצר
- קובץ הרונפיגורציה

אם אתם רוצים להריץ אופטימיזציה נוספת, פשוט שנו קונפיגורציה ותריצו שוב, התוצרים יישמרו בתיקייה נפרדת.

כמה הערות:

- כשיש צורך במספר "מתארים" עבור אותו מערך – ניתן להריץ מספר בדיד של קונפיגורציות כאלה (נניח לשמור כמה configs ולרוץ עליהם עם for בקריאה לrun_optimization_script.m) ויישמרו כלל התוצאות, ניתן להשתמש בהמשך בCSV שנוצרו בשביל לייצר lookup-table לכל אותם מקרים.
- השארתי את הסקריפט \MATLAB\compare_classical.m למקרה שיהיה לכם שימושי מתישהו, הוא משווה את העקום המתקבל למספר שיטות של tapering – זה היה סקריפט עזר שעזר לי לדבג את הפיתוח ובתכלס אין לו שימוש אמיתי מלבד בדיקת שפיות שהאלגוריתם מצליח יותר מהפתרון הקלאסי.
- שמתי גם תיקיית exe שבה יש קובץ הרצה (ואז על פניו לא נדרש שום toolbox) – אצלי זה עובד אבל זה יוצר ומשתמש בתיקייה בכונן אחר, תבדקו אם זה עובד אצלכם. זה סתם nice-to-have.